INGENIERÍA DE SOFTWARE I · ESPOL · PAO 2026-2

Control de lectura: SWEBOK sin dolor (o con poquito)

Definiciones claras, ejemplos que se te quedan y la página exacta del SWEBOK para cada cosa. Hecho para leerse de corrido una mañana, con café y algo de pánico controlado.



Fuente principal: **SWEBOK Guide v4.0a** (IEEE Computer Society), archivo *material/swebok-v4 (1).pdf* del vault.
Complemento: slides de clase 01b, 02, 03a (Dr. Mera / Dra. Villavicencio).
Preparado el 2026-10-20. Las definiciones están parafraseadas y traducidas, no copiadas. Los ejemplos con humor son inventados (ningún estudiante fue dañado… todavía).

ANTES DE EMPEZAR · 3 MINUTOS

Cómo usar esta guía

El profe lo dijo en la Sesión 03 (2026-10-05): va a preguntar *«según el SWEBOK, ¿qué es X?»*, quiere **2 o 3 líneas sin inventarse nada** y, para saber si entendiste, te va a pedir **un ejemplo**. Esta guía está armada exactamente para eso.

📖 Definición	🧠 En cristiano	🎯 Ejemplo
Lo que dice el SWEBOK, en español y en corto. Esto es lo que escribes en el control.	La misma idea como se la explicarías a tu pana a las 2 a. m.	El ejemplo concreto que te piden. Algunos con humor negro, porque así se recuerdan.

Fórmula de respuesta de 3 líneas (úsala en todas):
1) Qué es (la definición). **2)** Para qué sirve o en qué consiste (1 idea clave). **3)** Un ejemplo tuyo.
Ejemplo: «Según el SWEBOK, el mantenimiento es el conjunto de actividades para dar soporte costo-efectivo al software que ya está en operación. Incluye corregir, adaptar y mejorar el producto después de entregarlo. Ej.: actualizar la app del banco porque salió una nueva versión de Android (mantenimiento adaptativo).»

Cómo leer las referencias p. 46 (1-4)

El primer número es la **página del visor PDF** (la que pones en «ir a página»). Entre paréntesis va el número **impreso** en el libro (capítulo-página). Si el profe dice «capítulo 1, página 4», es la 1-4 = p. 46 del PDF.

Índice

1	La introducción del SWEBOK: software, ingeniería de software y por qué existe	p. 38–39
2	Mapa de las 18 áreas de conocimiento (KAs) en una línea cada una	toda la guía
3	KA 1, Requisitos de software: el plato fuerte	p. 43–61
4	KA 12, Calidad: error, defecto, falla, V&V	p. 246–257
5	KA 10 y 11: procesos, ciclos de vida, ágil y Scrum	p. 221–242
6	Las KAs que el profe nombró: SCM, mantenimiento, gestión, economía	p. 167–210, 288
7	Lo de clase que se cruza con el SWEBOK (taxonomía de Sommerville)	slides 03a
8	Trampas: parejas que se confunden	
9	Simulacro de control con respuestas modelo	
10	Chuleta de 1 página para el último repaso	

Lo que esta guía NO cubre (para que no te agarre de nuevo): la Lectura #3 de Canvas, *Mastering the Requirements Process* (Robertson), caps. 1, 10, 11, 14, 16 y 17. Ese libro no está en el vault y no te voy a inventar su contenido. Si el control también la incluye, toca leer esos capítulos aparte. Tampoco pude confirmar en Canvas el temario exacto del control: el token del Aula Virtual configurado está expirado.

SECCIÓN 1 · SWEBOK, INTRODUCCIÓN · P. 38–39

Lo básico: qué es todo esto

SWEBOK = *Software Engineering Body of Knowledge*, el «cuerpo de conocimiento» de la ingeniería de software. Es una guía de la IEEE Computer Society que dice qué tiene que saber un ingeniero de software. La v4 tiene **18 áreas de conocimiento** (KAs).

Software · software

DEFINICIÓN Programas de computadora, junto con los procedimientos y, posiblemente, la documentación y los datos asociados a la operación de un sistema de cómputo (vocabulario ISO/IEC/IEEE, citado por el SWEBOK).

EN CRISTIANO Software no es solo el código. Es el código **más** los manuales, los datos y los procedimientos para usarlo.

EJEMPLO El sistema de matrícula de la ESPOL es el código, la base de datos de materias y el instructivo en PDF que nadie lee hasta que se cae el servidor.

DÓNDE p. 38 (xxxvii) sección 1 «What Is Software Engineering?»

Ingeniería de software · software engineering PREGUNTA DE CAJÓN

DEFINICIÓN La aplicación de un enfoque **sistemático, disciplinado y cuantificable** al **desarrollo, la operación y el mantenimiento** del software; es decir, aplicar la ingeniería al software.

EN CRISTIANO No es «programar bonito». Es hacer software con método (sistemático), con reglas que se respetan (disciplinado) y midiendo (cuantificable). Y no termina cuando lo entregas: también cubre operarlo y mantenerlo.

EJEMPLO Programar el sistema del hospital un fin de semana con Red Bull es programación. Hacerlo con requisitos validados, pruebas, control de versiones, métricas de disponibilidad y un plan de mantenimiento es ingeniería de software. La diferencia la notan los pacientes.

DÓNDE p. 38 mismo párrafo. Palabras clave para memorizar: **sistemático · disciplinado · cuantificable · desarrollo · operación · mantenimiento**.

Ciencia vs. ingeniería

SWEBOK La ciencia descubre cosas nuevas. La ingeniería **aplica** ese conocimiento para resolver problemas reales con **recursos limitados** y de forma costo-efectiva.

EJEMPLO Un científico descubre un algoritmo de cifrado. El ingeniero decide si usarlo en la app del banco sabiendo que tiene 3 meses y 2 desarrolladores.

DÓNDE p. 39 (xxxviii)

Economía de la ingeniería · engineering economics

SWEBOK Analizar las soluciones teóricamente posibles para identificar la **más costo-efectiva**. Toda ingeniería la incluye.

EJEMPLO Se puede hacer la app en 5 lenguajes. Ingeniería es elegir el que da el mejor resultado por el dinero disponible, no el que está de moda en TikTok.

DÓNDE p. 39 y KA 15 p. 288 (15-1)

Área de conocimiento · knowledge area, KA

SWEBOK Un área identificada de la ingeniería de software, descrita por su conocimiento generalmente aceptado: procesos, prácticas, entradas, salidas, herramientas y técnicas.

DÓNDE p. 38

Qué cambió en la versión 4

- KAs **nuevas**: Arquitectura, Seguridad y Operaciones.
- **Ágil y DevOps** se incorporaron en casi todas las KAs.
- IA, machine learning e IoT entran en las KAs de fundamentos.

DÓNDE p. 38

«¿Por qué es importante la ingeniería de software? Dé 2 razones» DE CLASE

Esto sale de las slides y de Sommerville (cap. 1), no del SWEBOK: **(1)** la sociedad depende cada vez más de sistemas de software (bancos, salud, transporte), así que necesitamos producir software confiable de forma económica y rápida; **(2)** a largo plazo suele ser más barato usar métodos de ingeniería que programar «como salga», porque la mayor parte del costo está en cambiar el software después de que está en uso. Ejemplo: arreglar un requisito mal entendido cuesta un correo en la fase de requisitos y cuesta un rediseño (y un cliente furioso) en producción.

SECCIÓN 2 · LECTURA #0: «RECONOCER AL MENOS 8 ÁREAS»

Las 18 KAs en una línea

Si te preguntan «¿qué es *software X* según el SWEBOK?», casi siempre la respuesta está en el primer párrafo de la introducción de esa KA. Aquí está ese párrafo exprimido. Las 15 primeras son «de práctica»; las 3 últimas son «de fundamentos».

#	KA	Qué es (versión de control)	Ejemplo	Pág.
1	**Software Requirements** Requisitos	Desarrollar los requisitos del software y **gestionarlos durante toda su vida útil**, no solo al inicio.	Definir qué debe hacer la app de pedidos del bar y mantener ese acuerdo cuando el dueño cambie de idea (cada martes).	43
2	**Software Architecture** Arquitectura	La estructura de alto nivel del sistema: sus conceptos, cómo se describe, su proceso y cómo se evalúa. En la v4 se separó de diseño.	Decidir que el sistema es cliente-servidor con 3 capas antes de escribir una línea.	67
3	**Software Design** Diseño	Analizar los requisitos para definir las **características externas y la estructura interna** del software, como base para construirlo. Etapas: arquitectónico, alto nivel y detallado.	Diagramas UML de clases antes de programar el módulo de pagos.	83
4	**Software Construction** Construcción	Creación detallada del software mediante **codificación, verificación, pruebas unitarias, pruebas de integración y depuración**.	Escribir el código, probarlo y depurarlo a las 3 a. m. (lo de las 3 a. m. es opcional, pero tradicional).	99
5	**Software Testing** Pruebas	**Validación dinámica** de que el sistema da el comportamiento esperado en un **conjunto finito** de casos de prueba bien elegidos de un dominio de ejecución normalmente infinito.	No puedes probar todas las cédulas del país; eliges casos: válida, inválida, vacía, con letras.	117
6	**Software Engineering Operations** Operaciones · nueva	Actividades para **desplegar, operar y dar soporte** al software preservando su integridad y estabilidad, hasta que se retira. Aquí vive DevOps.	Monitorear que el sistema de notas no se caiga el día que publican las calificaciones.	151
7	**Software Maintenance** Mantenimiento	La **totalidad de actividades** para dar soporte costo-efectivo al software en operación (antes y después de entregarlo).	Parchar un bug, adaptar a la nueva versión de iOS, agregar una función.	167
8	**Software Configuration Management** Gestión de configuración, SCM	Aplicar gestión de configuración en todo el ciclo de vida para asegurar que los **ítems de configuración** estén completos y correctos: identificarlos, **controlar sus cambios**, registrar su estado y verificar que cumplen.	Git + ramas + versiones etiquetadas. Que nadie suba «final_final_ahora_si_v3.zip».	185
9	**Software Engineering Management** Gestión	Planificar, estimar, medir, controlar, coordinar, liderar y **gestionar riesgos** de un proyecto para entregar de forma eficiente y en beneficio de los stakeholders.	El cronograma del proyecto CogniSync y el plan B si un integrante desaparece.	202
10	**Software Engineering Process** Proceso	Las actividades de trabajo para construir y operar software: conceptos, **ciclos de vida** y evaluación/mejora del proceso.	Elegir cascada, incremental o Scrum para el proyecto.	221
11	**Models and Methods** Modelos y métodos	Modelos y métodos que dan estructura a la ingeniería de software para hacerla **sistemática, repetible y orientada al éxito**. Incluye métodos formales, prototipado y ágiles.	Un diagrama de estados para el préstamo de un libro; usar Scrum como método.	233
12	**Software Quality** Calidad	Grado en que el software cumple sus requisitos, siempre que esos requisitos representen de verdad lo que los stakeholders necesitan (ver sección 4).	Cumple todo lo pedido… pero lo pedido era lo incorrecto: no hay calidad.	246

13	**Software Security** Seguridad · nueva	Construir la seguridad **durante todo el ciclo de vida** (requisitos, diseño, construcción, pruebas y mantenimiento), no parcharla al final.	Cifrar contraseñas desde el diseño, no después de salir en las noticias.	264
14	**Professional Practice** Práctica profesional	Conocimientos, habilidades y actitudes para ejercer de forma **profesional, responsable y ética** (códigos ACM/IEEE, trabajo en equipo, comunicación).	No firmar que el sistema está probado cuando no lo está.	274
15	**Software Engineering Economics** Economía	Alinear las decisiones técnicas con los objetivos del negocio. «La economía es la ciencia de **elegir**, no del dinero». Pregunta clave: ¿conviene invertir aquí o rinde más en otro lado?	¿Desarrollar el sistema propio o comprar uno hecho (COTS)? Se comparan alternativas.	288
16	**Computing Foundations**	Base de ciencias de la computación que el ingeniero aplica. Distingue al ingeniero del simple programador.	Estructuras de datos, redes, sistemas operativos.	312
17	**Mathematical Foundations**	Lógica y razonamiento formal para traducir reglas sin ambigüedad a código.	Lógica proposicional, conjuntos, grafos.	345
18	**Engineering Foundations**	Lo común a todas las ingenierías: proceso de ingeniería, medición, toma de decisiones, análisis de causa raíz.	Empezar por «entender el problema real», que no siempre es el que te pidieron.	367

Truco para el «nombre 8 áreas»: sigue el ciclo de vida. **Requisitos** → **Arquitectura** → **Diseño** → **Construcción** → **Pruebas** → **Operaciones** → **Mantenimiento** (7) + **Configuración** (8). Ya pasaste. Si quieres lucirte: Gestión, Proceso, Calidad, Seguridad.

Ojo: el índice del vault decía que KA 1 empieza en p. 44; la portada del capítulo está en la **p. 43** (la definición de requisito sí está en la 44).

SECCIÓN 3 · KA 1 · P. 43–61 · EL CAPÍTULO CON MÁS PREGUNTAS

Requisitos de software

Es el tema que estás viendo en clase y en el proyecto. Si solo te da tiempo para una sección, que sea esta.

Las dos caras de los requisitos

SWEBOK Los requisitos se ven desde 2 perspectivas: **(1)** como expresión de las necesidades y restricciones de un producto o proyecto que ayudan a resolver un problema real; **(2)** como las **actividades** necesarias para desarrollarlos y mantenerlos.

EN CRISTIANO Requisito es «lo que hay que lograr» y también «el trabajo de averiguarlo y cuidarlo».

DÓNDE p. 43 (1-1) Introduction

Por qué duele hacer mal los requisitos

SWEBOK Cada requisito genera muchas decisiones de diseño; cada una, muchas de código; cada una, varias de pruebas. Un requisito faltante o mal entendido que no se detecta temprano provoca un **retrabajo que crece exponencialmente, en cascada**.

EJEMPLO Nadie preguntó si «fecha» era con hora. Tres meses después, todos los reportes del sistema de turnos médicos dan mal la hora de las citas. El requisito costaba una pregunta; ahora cuesta medio sistema.

DÓNDE p. 43

Incompletitud · incompleteness

SWEBOK Existen requisitos o detalles necesarios de los stakeholders que **no se revelan ni se comunican** a los ingenieros.

EJEMPLO El cliente «olvidó» decir que el 30 % de sus clientes no tiene correo.

Ambigüedad · ambiguity

SWEBOK El requisito se comunica de forma que admite **varias interpretaciones**, y solo una es la correcta.

EJEMPLO «Alerta rápida al médico». ¿Rápida para quién? El paciente y el programador no coinciden.

Son los **2 problemas principales** de requisitos en proyectos reales p. 43.

Defecto (bug) visto desde requisitos

SWEBOK Lo que llamamos «bug» se llama mejor **defecto**: una diferencia observable entre lo que el software **debería hacer** y lo que **hace**. Sin requisitos documentados, sabes qué hace el código, pero no qué *debía* hacer.

EJEMPLO Heredas un sistema sin documentación que redondea las notas hacia abajo. ¿Bug o política cruel del profe? Sin requisitos, no hay forma de saberlo.

DÓNDE p. 43

Requisito de software · software requirement DEFINICIÓN CLAVE

FORMAL (1) Condición o capacidad que un usuario necesita para resolver un problema o lograr un objetivo; (2) condición o capacidad que un sistema debe cumplir o tener para satisfacer un contrato, estándar, especificación u otro documento formal; (3) una representación documentada de (1) o (2).

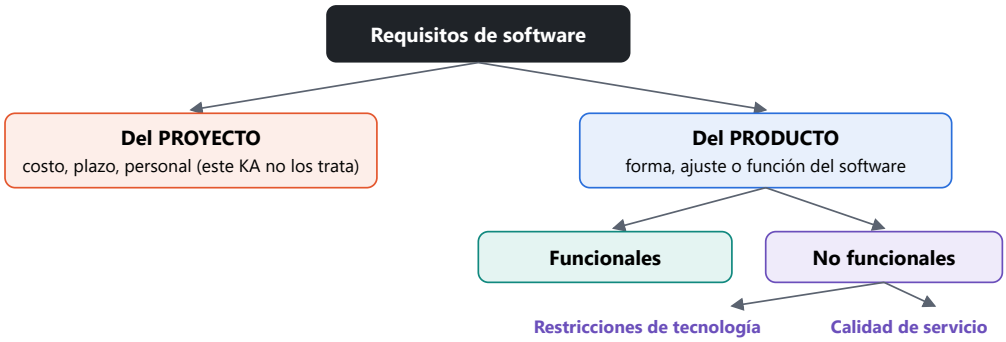
BÁSICA Una **propiedad que el software debe exhibir** para resolver un problema del mundo real.

EN CRISTIANO Algo que el sistema tiene que hacer o cumplir, porque alguien lo necesita o porque un papel firmado lo exige.

EJEMPLO «El sistema deberá impedir que un estudiante se matricule si tiene valores pendientes». Viene de una necesidad del negocio (que paguen) y es verificable.

DÓNDE p. 44 (1-2) sección 1.1 y p. 45 (1-3). Quién los impone: clientes, usuarios y también terceros como **entes reguladores** o el propio proyecto.

Las categorías de requisitos (figura 1.2, p. 46)



Redibujo simplificado de la figura 1.2 del SWEBOK. Ojo: es una clasificación distinta a la de Sommerville que viste en clase (ver sección 7).

Requisitos de producto vs. de proyecto

SWEBOK **Producto:** especifican la forma, ajuste o función esperada del software. **Proyecto** (o de proceso, o de negocio): restringen al proyecto que lo construye: costo, plazo, personal, ambiente de pruebas, migración de datos, capacitación.

EJEMPLO Producto: «calcular el promedio ponderado». Proyecto: «entregar el 2026-11-04 con 5 personas y presupuesto cero» (realismo universitario).

DÓNDE p. 45 (1-3) 1.3

Requisito funcional · functional requirement

SWEBOK Especifica **comportamientos observables** del software: las **políticas** que debe hacer cumplir y los **procesos** que debe llevar a cabo.

EJEMPLO Banco: política «el saldo nunca puede ser negativo»; proceso «transferir dinero entre cuentas». Del libro, p. 46.

DÓNDE p. 46 (1-4) 1.4

Requisito no funcional · nonfunctional requirement

SWEBOK Restringe de algún modo las **tecnologías** a usar en la implementación: ¿qué plataforma?, ¿qué base de datos?, ¿qué tan exactos o qué tan rápidos deben ser los resultados?, ¿cuántos registros guardar? Se dividen en **restricciones de tecnología** y **restricciones de calidad de servicio**. Se afectan entre sí: si cambias uno, revisa el impacto en los demás.

EJEMPLO «El sistema deberá responder una consulta de notas en menos de 2 s con 3 000 estudiantes conectados». No cambia *qué* hace, sino *qué tan bien*.

DÓNDE p. 46 (1-4) 1.5

Restricción de tecnología · technology constraint

SWEBOK Obliga a usar **o prohíbe** una tecnología específica y con nombre, o una infraestructura definida.

EJEMPLO «Debe correr en Android», «la base de datos será PostgreSQL», «prohibido usar punteros» (el libro da este último).

DÓNDE p. 46 1.6

Calidad de servicio · quality of service constraint

SWEBOK No nombra tecnologías: especifica los **niveles de desempeño aceptables** de la solución: tiempo de respuesta, throughput, exactitud, confiabilidad, escalabilidad. Lista de referencia: ISO/IEC 25010. Seguridad y safety se suelen olvidar.

EJEMPLO «99,5 % de disponibilidad en semana de exámenes».

DÓNDE p. 46–47 1.7

El filtro de la tecnología perfecta · Perfect Technology Filter

SWEBOK Para separar RF de RNF: imagina una computadora de **velocidad infinita, memoria ilimitada, costo cero y que nunca falla**. Lo que **todavía** tendrías que pedir es funcional. Todo lo demás es no funcional.

EJEMPLO Con la computadora perfecta igual necesitas «registrar la nota del estudiante» → RF. Pero «responder en 2 s» ya no tiene sentido (responde al instante) → RNF.

DÓNDE p. 47 (1-5) 1.8 «Why Categorize Requirements This Way?»

¿Para qué categorizar así? (1.8)

SWEBOK Cada categoría viene de **fuentes distintas** y se elicita, analiza, especifica y valida con técnicas y personas distintas. Además: (a) **divide y vencerás**: un problema grande se vuelve dos pequeños; (b) separa la experticia: el stakeholder sabe del negocio, el ingeniero sabe de tecnología. Un RNF en un dominio padre puede **inducir RF** en un dominio hijo (seguridad bancaria → funciones de autenticación).

EJEMPLO No le das a la contadora del cliente un documento que mezcla «calcular IVA» con «usar Kubernetes». Se rinde a la página 2.

DÓNDE p. 47

Requisitos de sistema vs. de software

SWEBOK Un **sistema** (def. INCOSE) es una combinación de elementos que interactúan para lograr un objetivo: hardware, software, firmware, personas, información, instalaciones… Los requisitos de sistema aplican al todo; los de software, solo a la parte de software. Algunos de software se **derivan** de los de sistema.

EJEMPLO Vehículo autónomo: «frenar ante un peatón» es del sistema; «detectar el peatón en la imagen en <50 ms» es del software.

DÓNDE p. 47–48 1.9

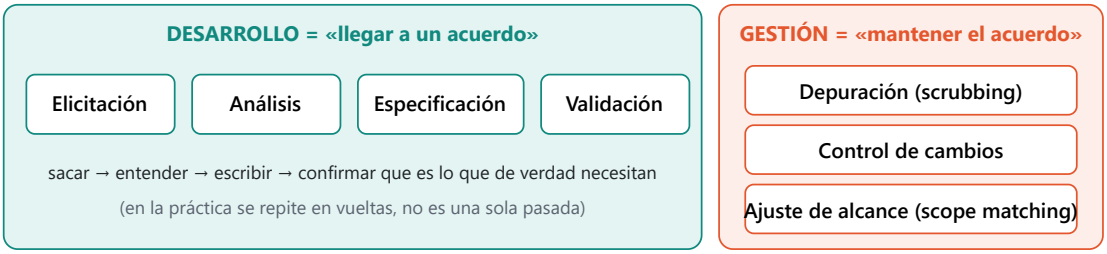
Requisito derivado · derived requirement

SWEBOK Requisito que **no** lo pone un stakeholder externo, sino que se impone **dentro** del equipo de desarrollo. Depende de la perspectiva: para el cliente es una decisión de diseño; para el subequipo es un requisito.

EJEMPLO El arquitecto decide «tubos y filtros». Para el cliente da igual; para el equipo que hace un filtro, es ley.

DÓNDE p. 48 (1-6) 1.10

Las actividades de requisitos (figura 1.3, p. 48)



Desarrollo de requisitos = ponerse de acuerdo en qué se construye. Gestión = mantener ese acuerdo en el tiempo. p. 48 (1-6) 1.11

KA 1 · 2. ELICITACIÓN · P. 48–50

Elicitación: sacarle los requisitos a la gente

Elicitación · requirements elicitation

SWEBOK Su meta es **hacer aflorar requisitos candidatos**. También se llama captura, descubrimiento o adquisición de requisitos. Una buena elicitación **reduce la incompletitud**, aunque nunca garantiza que el conjunto esté completo. **No es una actividad pasiva**: los usuarios no saben describir su trabajo, se callan cosas o no quieren colaborar, y muchos requisitos son tácitos.

EJEMPLO Le preguntas a la secretaria cómo registra a los pacientes y te dice «fácil, en el sistema». Observándola descubres que lleva un cuaderno paralelo porque el sistema «a veces se come los datos». Eso no te lo iba a decir nunca.

DÓNDE p. 48 (1-6) sección 2

Stakeholder (interesado) PREGUNTA PROBABLE

SWEBOK Cualquier persona, grupo u organización que: **(1)** participa activamente en el proyecto, **(2)** es **afectado** por su resultado, o **(3)** puede **influir** en su resultado.

TIPOS **Clientes** (pagan) · **customers** (deciden si se pone en servicio) · **usuarios** (interactúan directa o indirectamente; se dividen en clases de usuario) · expertos del dominio (SMEs) · personal de operaciones y soporte · gremios profesionales · **entes reguladores** · grupos de interés · **gente perjudicada si el proyecto tiene éxito** · desarrolladores.

EJEMPLO Sistema de autoservicio en un supermercado: el gerente paga (cliente), el cajero lo usa… y también es el que podría quedarse sin trabajo si funciona bien. Sí, el SWEBOK incluye explícitamente a los perjudicados. Nadie dijo que la ingeniería fuera tierna.

DÓNDE p. 48–49 2.1 «Requirements Sources»

Clases de stakeholders y análisis de stakeholders

SWEBOK Una **clase** agrupa stakeholders con perspectivas y necesidades parecidas. El **análisis de stakeholders** busca identificar todas las clases importantes para que los requisitos no queden sesgados hacia los que más hablan, y ayuda a negociar conflictos.

EJEMPLO En la app de la ESPOL, si solo entrevistas a los profes, la app servirá para poner notas y no para revisarlas.

DÓNDE p. 49 (1-7)

Fuentes que no son personas

SWEBOK Documentación (versiones anteriores, misión, concepto de operaciones), **otros sistemas**, el contexto de negocio (políticas y procesos) y el ambiente de cómputo.

EJEMPLO El reglamento de la ESPOL dice cuántas veces puedes repetir una materia: requisito que no salió de ninguna entrevista.

DÓNDE p. 49

Técnicas de elicitación p. 49–50 (1-7 a 1-8) 2.2

Con stakeholders

- **Entrevistas**
- **Reuniones**, con lluvia de ideas
- **JAD / JRP** y talleres facilitados
- Análisis de protocolo
- **Grupos focales**
- **Cuestionarios** y encuestas de mercado
- **Prototipado exploratorio** (baja y alta fidelidad)
- **User story mapping**

Sin stakeholders (u otras fuentes)

- Versiones previas y su base de defectos
- Sistemas que se conectan con el nuevo
- Benchmarking de la competencia, búsqueda de literatura
- QFD (casa de la calidad)
- **Observación** del trabajo en su ambiente
- **Aprendizaje** (*apprenticing*): haces el trabajo para aprenderlo
- Escenarios de uso, análisis de tareas, **descomposición** (capacidades → épicas → features → historias)
- **Design thinking**: empatizar, definir, idear, prototipar, probar
- ISO 25010, requisitos de seguridad, normas y regulaciones

Ejemplo para el control: «Apprenticing: el ingeniero aprende haciendo el trabajo. Ej.: para hacer el sistema de una panadería, trabajo dos madrugadas amasando y descubro que el pedido de harina depende del clima, cosa que el dueño nunca mencionó.» (Bonus: no hay mejor forma de entender por qué el panadero odia el software).

KA 1 · 3. ANÁLISIS · P. 50–52

Análisis: entender lo que te dijeron

Análisis de requisitos · requirements analysis

SWEBOK Los requisitos casi nunca salen de la elicitación en su forma final. El análisis ayuda a entender el **significado y las implicaciones** de los requisitos candidatos, uno por uno y en el contexto de todo el conjunto.

DÓNDE p. 50 (1-8) sección 3

Propiedades deseables de un requisito MUY PREGUNTABLE

Cada requisito debe ser:

- **No ambiguo**: una sola interpretación
- **Verificable (cuantificado)**: se puede demostrar si se cumple
- **Vinculante**: el cliente pagaría por él y no quiere quedarse sin él
- **Atómico**: una sola decisión
- Representar necesidades **reales**
- Usar el **vocabulario del stakeholder**
- **Aceptable** para todos los stakeholders

El conjunto debe ser:

- **Completo**: cubre límites, excepciones y seguridad
- **Conciso**: sin relleno
- **Consistente internamente**: no se contradicen entre sí
- **Consistente externamente**: no contradicen las fuentes
- **Factible**: se puede hacer dentro de costo, plazo y personal

EJEMPLO «El sistema deberá ser rápido y fácil y enviar correos» falla en 3: es ambiguo («rápido»), no verificable y no es atómico (son 3 decisiones en una).

DÓNDE p. 50 3.1 «Basic Requirements Analysis»

Los 5 porqués · 5-whys

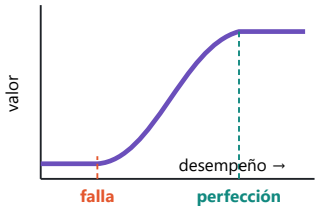
SWEBOK A veces lo que se elicita es una **solución**, no el problema real. Se pregunta repetidamente «¿por qué es este el requisito?» hasta llegar a «si no se hace eso, el problema del stakeholder no se resuelve». Normalmente basta con 2 o 3 vueltas; se llama «5» para que insistas.

EJEMPLO «Quiero un botón para exportar a Excel». ¿Por qué? Para mandar el reporte al jefe. ¿Por qué? Porque el jefe no tiene acceso al sistema. Problema real: **darle acceso al jefe**. El botón era la solución equivocada (y el Excel, una tragedia anunciada).

DÓNDE p. 50

Economía de la calidad de servicio: punto de falla y punto de perfección

SWEBOK Para un RNF de calidad de servicio, el valor sube con el nivel de desempeño entre dos puntos: el **punto de falla** (peor nivel aceptable; por debajo ya no vale nada) y el **punto de perfección** (mejor nivel útil; por encima no aporta más). El costo sube en escalones. El nivel **más costo-efectivo** es donde la diferencia valor − costo es máxima. Además, unos RNF se ayudan entre sí (código mantenible suele ser más confiable) y otros se pelean (más velocidad suele dar menos mantenibilidad).



EJEMPLO Una red social que soporte más usuarios que la población mundial ya pasó su punto de perfección (ejemplo del propio libro). Pagar por eso es tirar plata.

DÓNDE p. 50–51 (1-8, 1-9) 3.2, figuras 1.4 y 1.5

Análisis formal

SWEBOK Expresar requisitos en un **lenguaje con semántica formal**. Ventajas: son precisos y concisos (menos malinterpretación) y se puede **razonar y demostrar** propiedades (p. ej., ausencia de deadlock). Útil en sistemas de alta integridad.

EJEMPLO El software de un marcapasos. Ahí «más o menos funciona» no es una opción.

DÓNDE p. 51–52 3.3

Manejo de conflictos

SWEBOK Dos enfoques: **(1) negociar** entre stakeholders (no decidir unilateralmente; que quede trazable al cliente); **(2) familia de productos**: separar requisitos **invariantes** (todos de acuerdo) de **variantes** (en conflicto) y diseñar puntos de configuración.

EJEMPLO Unos quieren °C y otros °F: no peleas, pones una opción de configuración (ejemplo del libro).

DÓNDE p. 52 (1-10) 3.4

KA 1 · 4. ESPECIFICACIÓN · P. 52–57

Especificación: dejarlo por escrito

Especificación de requisitos · requirements specification

SWEBOK Registrar los requisitos para que se puedan **recordar y comunicar**. Es el tema más polémico del KA: ¿se escriben?, ¿en qué forma?, ¿se mantienen? No hay respuesta estándar: depende del riesgo, la rotación de personal, si el equipo está distribuido, si se terceriza, contratos, etc. Recomendación básica: hacer un **análisis de audiencia** (quién lee qué y cómo dárselo con el menor esfuerzo).

IDEA FINA El **qué** y el **cómo** del trabajo de requisitos los define la naturaleza del software, **no el ciclo de vida**. Un mantenedor futuro no debería poder adivinar si usaron cascada o Scrum mirando los requisitos. En cascada están completos al final de la fase de requisitos; en ágil cambian y crecen hasta el final.

EJEMPLO La startup que «no necesita documentar» funciona hasta que el único que sabía por qué existe el botón rojo se va a Canadá.

DÓNDE p. 52–53 (1-10, 1-11) sección 4. Plantillas: ISO/IEC/IEEE 29148 (la misma de tu proyecto parcial).

Técnica de especificación	Qué es	Ejemplo	Pág.
Lenguaje natural no estructurado	Frases comunes tipo «El sistema deberá…». Incluye **reglas de negocio**: frases que definen o restringen la estructura o el comportamiento del negocio.	«Un estudiante no puede matricularse si tiene valores pendientes» (del libro).	53
Lenguaje natural estructurado	Impone una forma para ganar precisión. El más simple: **actor-acción** = [evento disparador] + actor + acción + [condición]. También plantillas de **casos de uso**, **historias de usuario** y tablas de decisión.	«Cuando se despacha un pedido, el sistema deberá crear una factura, salvo que sea prepagado».	54
Basada en criterios de aceptación (ATDD y BDD)	Escribir los requisitos **como casos de prueba**. Ataca directamente la **ambigüedad** (el lenguaje de prueba es preciso), pero no resuelve sola la incompletitud.	Ver BDD abajo.	54–55
Basada en modelos	Usar UML o SysML como «planos». **Estructurales** (modelo de clases, entidad-relación) para políticas; **de comportamiento** (casos de uso, estados, actividades, flujo de datos) para procesos. Formalidad: ágil (bocetos) < semiformal < formal (Z, VDM).	Diagrama de estados de un préstamo: activo → vencido → devuelto.	56

Historia de usuario (formato SWEBOK)

> Como <rol> quiero <capacidad> para <beneficio>

EJEMPLO «Como estudiante con 3 parciales la misma semana, quiero ver todas mis fechas en un calendario para planificar mi colapso nervioso con anticipación».

DÓNDE p. 54 (1-12) 4.2 y 4.3

Escenario BDD

> Dado <contexto>, cuando <estímulo>, entonces <resultado>

EJEMPLO Dado saldo de $50 y tarjeta válida, cuando pido $100 en el cajero, entonces no entrega dinero, avisa saldo insuficiente, el saldo sigue en $50 y devuelve la tarjeta (ejemplo del libro, p. 55).

ATDD en 3 pasos · acceptance test driven development

SWEBOK Parte de TDD (las pruebas van antes del código). **1)** Se elige una unidad de funcionalidad (p. ej., una historia). **2)** Ingenieros, expertos del negocio y QA acuerdan, antes de programar, las pruebas que deben pasar. **3)** Al menos una debe fallar con el software actual; eso da «permiso» para programar hasta que todas pasen. Truco: cambia «esperamos que el software produzca Y» por «el software deberá producir Y» y la prueba se vuelve un requisito.

DÓNDE p. 54–55. BDD es la versión más amigable para el negocio (historias + escenarios).

Atributos adicionales

SWEBOK Además del texto, documentar: etiqueta para trazabilidad, descripción, **justificación**, **fuente**, tipo, dependencias, conflictos, **criterios de aceptación**, **prioridad**, **estabilidad**, historial de cambios…

DÓNDE p. 56–57 4.5

Especificación incremental vs. integral

SWEBOK **Incremental:** cada versión solo trae los cambios (menos papel). **Integral** (*comprehensive*): cada versión trae todos los requisitos (un solo documento para entender todo). Se combinan: versiones x.1 incrementales, versiones 1.0, 2.0 integrales.

DÓNDE p. 57 (1-15) 4.6

KA 1 · 5–7. VALIDACIÓN, GESTIÓN Y PRÁCTICA · P. 57–61

Validar, gestionar y no morir en el intento

Validación de requisitos · requirements validation

SWEBOK Ganar **confianza** en que los requisitos representan las **verdaderas necesidades** de los stakeholders, tal como se entienden hoy. Preguntas: ¿están todos?, ¿hay alguno que no representa una necesidad real?, ¿están bien escritos, son entendibles, consistentes y completos?, ¿cumplen los estándares? **3 métodos:** revisiones, simulación/ejecución y prototipado.

DÓNDE p. 57 (1-15) sección 5

1. Revisiones

La forma más común: revisores buscan errores, omisiones, supuestos inválidos y falta de claridad. Mejor desde varias perspectivas: usuarios, expertos en requisitos y quienes van a diseñar. Ayuda dar checklists o una «definition of done».
Ej.: la ayudante «abogada del diablo» revisando tu ERS.
p. 57–58

2. Simulación y ejecución

Si la especificación es formal o basada en modelos, se «ejecuta» con escenarios para convencer a stakeholders que no quieren leer documentos.
Ej.: recorrer el diagrama de estados del pedido con el dueño del local, paso a paso.
p. 58

3. Prototipado

Un prototipo muestra cómo interpretó el ingeniero los requisitos y expone sus supuestos. **Riesgo:** lo cosmético distrae de la funcionalidad, y puede ser caro.
Ej.: el cliente pasa 40 min discutiendo el color del botón y 0 min viendo si el cálculo está bien.
p. 58 (1-16)

Gestión de requisitos p. 58–59 sección 6

Depuración (scrubbing)

Encontrar el **conjunto más pequeño** de requisitos simples que satisfaga las necesidades: eliminar los que están fuera de alcance, no dan retorno o no son tan importantes, y simplificar los complicados. En ágil ocurre en la planificación del sprint.
Ej.: «modo oscuro con IA que detecta tu estado de ánimo» → fuera.

Control de cambios

En procesos por plan: forma de pedir cambios, análisis de impacto (opcional), un responsable que acepta/rechaza/difiere, notificar y dar seguimiento. Aceptar un cambio = aceptar su impacto en plazo y recursos. En ágil: el cambio es un ítem más del **product backlog** y se «acepta» cuando entra a un sprint.
p. 59 (1-17)

Ajuste de alcance (scope matching)

Asegurar que el alcance no supere costo, plazo o personal. Si lo supera: reducir alcance (quitar lo de menor prioridad), aumentar capacidad, o negociar. En ágil: con planificación por velocidad solo entra al sprint lo que se puede terminar.
p. 59

Consideraciones prácticas p. 59–61 sección 7

Priorización y el modelo de Kano

SWEBOK Priorizar sirve para entregar primero lo más valioso, resolver conflictos y ajustar alcance. Factores: valor, insatisfacción si falta, costo de entregar y mantener, riesgo técnico, riesgo de que nadie lo use. El **modelo de Kano** dice que no basta con ver cuánto gusta algo: hay que ver cuánto **enoja que falte**. Fórmula de ejemplo: prioridad = valor × (1 − riesgo) / costo. Se comunica con escalas (must/should/nice to have), números o listas ordenadas.

EJEMPLO Cliente de correo: el filtro de spam gusta más, pero no poder abrir adjuntos enoja muchísimo más → los adjuntos van primero (ejemplo del libro, p. 60).

DÓNDE p. 59–60 (1-17, 1-18) 7.2

Proceso iterativo (7.1)

Los requisitos tienen amplitud y profundidad; es casi imposible hacerlos en una sola pasada.
p. 59

Trazabilidad (7.3)

Dos usos: (1) comprobar consistencia: ¿cada requisito tiene diseño que lo cumpla?, ¿cada elemento de diseño tiene un requisito que lo justifique?; (2) **análisis de impacto** de un cambio (requisito → diseño → código → pruebas). Hacia atrás (a las fuentes) y hacia adelante.
p. 60–61

Estabilidad y volatilidad (7.4)

Unos requisitos casi nunca cambian (calcular intereses en un banco) y otros sí (cuentas libres de impuestos, que dependen de una ley). Identificar los volátiles ayuda a diseñar para el cambio.
p. 61 (1-19)

Medir requisitos (7.5) p. 61**:** la **medición de tamaño funcional (FSM)** evalúa el tamaño de un conjunto de requisitos funcionales; los **story points** también cuentan como medida de tamaño. Sirve para estimar costo y el tamaño de un cambio.

SECCIÓN 4 · KA 12 · P. 246–257

Calidad de software

El profe la nombró textual: «¿qué es software quality en el contexto de ingeniería de software? 2 o 3 líneas».

Calidad de software · software quality LA NOMBRÓ EL PROFE

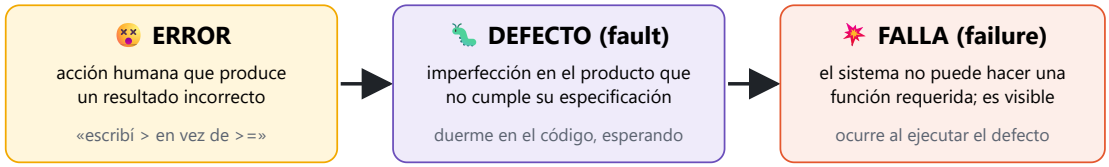
SWEBOK El **grado en que un producto de software cumple los requisitos establecidos**; pero la calidad depende de qué tan bien esos requisitos **representan las necesidades, deseos y expectativas** de los stakeholders. Otra definición: la capacidad del producto de satisfacer necesidades **declaradas e implícitas** bajo condiciones especificadas.

MATIZ El término está «sobrecargado»: puede referirse a las características deseables del producto, a qué tanto las tiene (**calidad del producto**) o a los procesos para lograrlas (**calidad del proceso**). Históricamente: Crosby, «conformidad con los requisitos»; Humphrey, «aptitud para el uso» (*fitness for use*); IBM, «el cliente es el árbitro final».

EJEMPLO Un sistema que cumple el 100 % de los requisitos, pero los requisitos olvidaron a los usuarios con discapacidad visual: cumple el papel, pero no tiene la calidad que la gente necesitaba. Calidad = cumplir **lo correcto**.

DÓNDE p. 246 (12-1) Introduction y p. 248 (12-3) 1. Fundamentals

Error → defecto → falla (la cadena del desastre) p. 248



El error lo comete una persona; el defecto queda en el software; la falla es lo que ve el usuario cuando se ejecuta el defecto.

Ejemplo completo para el control: a las 3 a. m. el programador escribe `nota > 6` en vez de `nota >= 6` (**error**). Esa línea queda en el sistema (**defecto**) y nadie lo nota durante meses. El día que sale la nota final, todos los que sacaron exactamente 6 aparecen reprobados (**falla**). Moraleja: el defecto existía desde siempre; la falla solo esperaba el peor día posible.

Costo de la calidad · CoSQ

SWEBOK Suma de costos de implementación, **prevención** (mejora de procesos, herramientas, capacitación), **evaluación** (revisiones, auditorías, pruebas) y **no conformidad** (retrabajo por fallas internas y externas). Se agrupa en costo de **conformidad** (prevenir + detectar) y de **no conformidad** (arreglar). Meta: el CoSQ **mínimo** para un nivel de calidad dado.

EJEMPLO Ahorrarte las pruebas «para ir más rápido» es como ahorrarte el freno para que el carro pese menos.

DÓNDE p. 249 (12-4) 1.2

Verificación vs. validación (V&V)

SWEBOK **Verificación:** asegura que el producto se construye **correctamente**: lo que sale de una fase cumple lo que se le impuso al inicio de esa fase. **Validación:** asegura que se construye el **producto correcto**: cumple su propósito previsto.

EJEMPLO Construir perfecto, según los planos, un puente que va al lugar equivocado: verificado ✅, validado ❌.

DÓNDE p. 257 (12-12)

Dependability y sistemas críticos (safety-critical)

SWEBOK Los sistemas **críticos para la seguridad** son aquellos donde una falla puede dañar vidas, seres vivos, estructuras o el ambiente; exigen métodos sistemáticos y estándares (p. ej., DO-178C en aviación, EN 50128 en trenes). Tres técnicas para reducir el riesgo de falla: **evitar**, **detectar y eliminar**, y **limitar el daño**. Más riesgo = más SQA e inspecciones más rigurosas.

EJEMPLO El software de frenado de un tren: aquí «se cayó el sistema» no significa «recarga la página».

DÓNDE p. 250 (12-5) 1.4 «Software Dependability and Integrity Levels»

SECCIÓN 5 · KA 10 Y KA 11 · P. 221–242

Procesos, ciclos de vida y ágil

Proceso, actividad y tarea

SWEBOK **Proceso:** conjunto de actividades relacionadas que **transforman entradas en salidas**. **Actividad:** conjunto cohesivo de tareas de un proceso. **Tarea:** acción requerida, recomendada o permitida que contribuye a lograr un resultado del proceso. La salida de un proceso puede ser la entrada de otro.

EJEMPLO Proceso: pruebas (entrada: código; salida: reporte de defectos). Actividad: pruebas de integración. Tarea: probar que el login se conecta con la base de datos.

DÓNDE p. 223 (10-3) 1.2. Coincide con la jerarquía actividad → acción → tarea que viste en clase.

Ciclo de vida · life cycle

SWEBOK La **evolución** de un sistema, producto, servicio o proyecto **desde su concepción hasta su retiro**. Un ciclo de vida se crea conforme a un **modelo de ciclo de vida** (cascada, V, incremental, espiral, ágil). Etapas típicas: concepto, desarrollo, producción, utilización, soporte y retiro.

EJEMPLO El sistema académico nace en una reunión (concepto), se construye, se usa por 15 años, recibe parches y un día lo apagan sin que nadie llore. Ciclo de vida completo.

DÓNDE p. 223 2.1, p. 225 2.3, etapas en p. 227 (10-7)

Tipos de ciclo de vida p. 225 (10-5) 2.4

Tipo	Idea clave (SWEBOK)	Ejemplo
Predictivo	Alcance, tiempo y costo se fijan al **inicio**. Supone que los requisitos son un conjunto cerrado que no cambia «salvo fuerza mayor».	Software del controlador de un ascensor con especificación firmada.
Iterativo	El alcance se fija temprano, pero tiempo y costo se ajustan a medida que se entiende el producto. Ciclos repetidos.	Cada ciclo mejoras la misma app según lo aprendido.
Evolutivo	El producto cambia a lo largo de su vida, porque cambian las necesidades o porque los requisitos entran en pasos sucesivos.	Una red social: nunca está «terminada».
Incremental	El entregable se produce en iteraciones que **agregan funcionalidad**; solo está completo tras la última. Puede ser predictivo.	V1: buscar libros. V2: renovar. V3: notificaciones.
Desarrollo continuo	Prácticas que permiten liberaciones frecuentes a ambientes de prueba con herramientas automáticas.	CI/CD que despliega cada commit a staging.

Un ciclo de vida es **abierto al cambio** si permite cambiar requisitos incluso después de firmados (una de las banderas de ágil). Los predictivos lo prohíben una vez cerrada la fase de requisitos p. 225.

Los modelos clásicos según el SWEBOK p. 226 (10-6) 2.5

Cascada (waterfall)

Predictivo y **dirigido por documentos**. Fases en orden estricto (requisitos, diseño preliminar, detallado, código, pruebas); una no empieza hasta terminar la anterior, con poca o ninguna iteración. Aportó sistematización y fue un primer intento de responder a la **crisis del software**.
Variante: **modelo V**.
Ej.: perfecto si nada cambia. O sea, en otro universo.

Espiral (Boehm)

Evolutivo y dirigido por riesgos (no por documentos ni código). Las actividades (análisis, diseño, código, integración, pruebas) se repiten iterativamente hasta terminar el software.
Ej.: sistema de control de un satélite: cada vuelta ataca primero lo que más miedo da.

Prototipado rápido

Se desarrollan prototipos **temprano** para obtener retroalimentación y análisis desde el inicio.
Ej.: maqueta en Figma de la app antes de programar.

Proceso Unificado / RUP

Marco de desarrollo **iterativo e incremental**. RUP y OpenUP vienen del Proceso Unificado.
Ej.: proyecto grande que necesita iterar pero con gobernanza.

Manifiesto Ágil y los 3 mitos · Agile Manifesto

SWEBOK Cambió la mentalidad: el proceso debe estar **abierto al cambio**, con comunicación cara a cara, confianza equipo-cliente y **entregas pequeñas** en vez de una sola al final. Distingue **valores y principios** (entregar valor, excelencia técnica) de **prácticas** (pair programming, sprint planning, retrospectiva). **El manifiesto NO dice que no se documente**.

3 MITOS ❌ «Ágil es un método» (no lo es; Scrum y XP sí son métodos) · ❌ «Ágil es más rápido porque no se documenta» · ❌ «Ágil tiene pocas prácticas o es desordenado».

DÓNDE p. 226–227 (10-6, 10-7)

DevOps

SWEBOK Conjunto de principios y prácticas que mejoran la **comunicación y colaboración** entre stakeholders para especificar, desarrollar y **operar** software, con mejora continua en todo el ciclo de vida. Rompe los «silos» entre desarrollo y operaciones.

EJEMPLO Antes: «en mi máquina funciona» vs. «en producción no». Con DevOps, los dos equipos comparten la culpa… digo, los procesos.

DÓNDE p. 227; también KA 6 p. 151

Prototipado como método (KA 11)

SWEBOK Crear versiones incompletas o mínimas para probar ideas o pedir retroalimentación. Empieza por lo **menos entendido** (al revés que otros métodos). Estilos: desechable o en papel, evolutivo, o especificación ejecutable. Normalmente el prototipo **no** se vuelve el producto final sin mucho retrabajo.

DÓNDE p. 241 (11-9) 4.3

Métodos ágiles y Scrum, versión SWEBOK

Métodos ágiles

SWEBOK Surgieron en los años 90 para reducir la carga de los métodos pesados basados en planes. Son «ligeros» por: **ciclos cortos e iterativos**, **equipos autoorganizados**, diseño simple, refactorización, TDD, **participación frecuente del cliente** y un **producto funcionando** en cada ciclo. Se pueden ver como el ciclo de Deming **PDCA** (planificar-hacer-verificar-actuar) aplicado al software.

DÓNDE p. 241–242 (11-9, 11-10)

Scrum según el SWEBOK

SWEBOK El enfoque ágil **más amigable con la gestión de proyectos**. El **Scrum Master** gestiona las actividades dentro del incremento. Cada incremento es un **sprint** de **no más de 30 días**. El **Product Owner** decide qué entra al **product backlog**; sus tareas se identifican, definen, priorizan y estiman. En cada incremento se prueba y libera una versión funcionando. El **Daily Scrum** asegura que el trabajo siga el plan.

EJEMPLO Proyecto CogniSync: sprints de 2 semanas, el PO ordena el backlog, cada mañana 15 minutos de «qué hice, qué haré, qué me bloquea» (el bloqueo suele llamarse «el compañero que no contesta el WhatsApp»).

DÓNDE p. 242 (11-10)

Método	Qué lo distingue (SWEBOK)	Ejemplo
RAD	Para sistemas de negocio con muchos datos; usa herramientas especiales de bases de datos para desarrollar y desplegar rápido.	Un sistema de inventario armado con herramientas low-code.
XP	Historias para requisitos, **pruebas primero**, cliente dentro del equipo, **programación en parejas**, refactorización e integración continua. Entregas cada par de semanas.	Dos programadores, un teclado y una amistad a prueba de fuego.
Scrum	Sprints ≤ 30 días, Scrum Master, Product Owner, backlog, Daily Scrum.	Ver arriba.
FDD	Dirigido por modelos, 5 fases (modelo del producto, lista de features, plan, diseño por feature, codificar-probar-integrar). El código tiene **dueño individual** y pesa más la arquitectura.	Cada feature tiene un responsable con nombre y apellido (y culpa asignada).
Lean	Principios de manufactura de Toyota: **Producto Mínimo Viable**, aprender del usuario e iterar; optimizar todo el flujo de valor. **Kanban** aplica ideas Lean para visualizar y manejar el flujo.	Lanzar la app solo con «pedir café» y ver si alguien la usa antes de añadir el resto.

Lo que agrega la clase sobre Scrum SLIDES 03B / GUÍA: 3 roles (Product Owner, Scrum Master, equipo de desarrollo de 5–9 personas según las slides), 3 artefactos (product backlog, sprint backlog, incremento potencialmente entregable), eventos (sprint, planning, daily de 15 min, review, retrospectiva), tareas de ≤ 16 h, burndown y **velocidad = puntos de historia de lo que quedó «Done»** (una historia al 90 % aporta 0). Si preguntan «según el SWEBOK», usa la tarjeta de arriba; si preguntan por la clase, usa esto.

Trampa: el SWEBOK dice que el Scrum Master «gestiona las actividades»; la Guía Scrum moderna lo describe como facilitador que no asigna tareas. Si te preguntan *según el SWEBOK*, responde con la versión del SWEBOK.

SECCIÓN 6 · LAS KAS QUE EL PROFE NOMBRÓ EN CLASE

SCM, mantenimiento, gestión y economía

En la Sesión 03 mencionó explícitamente *configuration management*, *software quality* y *software engineering economics*. Calidad ya la viste; aquí van las otras con un poco más de detalle.

Gestión de configuración del software · SCM

LA NOMBRÓ EL PROFE

SWEBOK El proceso de aplicar la gestión de configuración **durante todo el ciclo de vida** para asegurar la **completitud y corrección** de los **ítems de configuración** (CIs). La gestión de configuración es la disciplina de: **identificar y documentar** las características de un CI, **controlar sus cambios**, **registrar y reportar** el estado de los cambios, y **verificar** que cumple los requisitos. Apoya a la gestión del proyecto, desarrollo, mantenimiento y aseguramiento de calidad.

EN CRISTIANO Saber exactamente qué versión de cada cosa (código, documentos, pruebas) tienes, quién la cambió, por qué, y poder volver atrás.

EJEMPLO En el proyecto, el ERS y el código están en Git; la versión entregada el 2026-11-04 queda etiquetada como v1.0. Cuando el cliente jure que «eso no lo pidió», abres el historial y le enseñas el commit con su correo de aprobación. Venganza documentada.

DÓNDE p. 185 (8-1) Introduction

Línea base · baseline

SWEBOK Versión **formalmente aprobada** de un CI, fijada en un momento de su ciclo de vida. Solo se puede cambiar mediante **control de cambios formal**.

EJEMPLO El ERS firmado por el cliente. Cambiarlo ya no es «un detallito», es una solicitud de cambio.

DÓNDE p. 191 (8-7) 2.3

Mantenimiento de software · software maintenance

SWEBOK La **totalidad de actividades** necesarias para dar **soporte costo-efectivo** al software en operación. Ocurre antes de la entrega (planificar, mantenibilidad, logística) y después (modificar, capacitar, soporte).

DÓNDE p. 167 (7-1)

Los tipos de mantenimiento p. 170–171 (7-4, 7-5)

Tipo	Qué es	Ejemplo
Correctivo	Reparación **reactiva** después de la entrega para corregir problemas descubiertos.	Arreglar el bug que reprobaba a los que sacaron 6.
Preventivo	Corregir **fallas latentes** antes de que se manifiesten en producción.	Ver que el campo «año» explota en 2038 y arreglarlo hoy.
Adaptativo	Mantener el software usable en un **ambiente que cambia**.	Adaptar la app a la nueva versión de Android.
Aditivo	Agregar funciones o features **relativamente grandes** después de la entrega.	Añadir pagos en línea a la app de reservas.
Perfectivo	Mejoras para usuarios, mejor documentación, recodificar para mejorar rendimiento o mantenibilidad.	Refactorizar el módulo que solo entiende el que ya renunció.
De emergencia	Modificación **no programada** para mantener el sistema operando temporalmente, mientras llega la corrección.	Parche a medianoche porque el sistema de pagos se cayó el día de la quincena.

ISO/IEC/IEEE 14764 agrupa correctivo y preventivo como **correcciones**; adaptativo, aditivo y perfectivo como **mejoras** (figura 7.2).

Gestión de ingeniería de software · SEM

SWEBOK Conjunto de actividades de **planificar, estimar, medir, controlar, coordinar, liderar y gestionar riesgos** de un proyecto, para entregar productos y servicios de forma eficiente, eficaz y en beneficio de los stakeholders. El software es **intangible y maleable**, por eso se gestiona distinto a lo físico.

DÓNDE p. 202 (9-1)

Riesgo · risk

SWEBOK La **incertidumbre** es falta de información; el **riesgo** es el efecto de la incertidumbre sobre los objetivos, con consecuencias negativas (amenazas) **o positivas** (oportunidades). Gestionarlo = identificar, analizar probabilidad e impacto, priorizar y mitigar.

EJEMPLO Riesgo: el cliente del proyecto deja de responder. Mitigación: tener su aprobación por escrito de cada entrega.

DÓNDE p. 210 (9-9) 2.5

Economía de la ingeniería de software · software engineering economics LA NOMBRÓ EL PROFE

SWEBOK Ayuda a que las decisiones técnicas se **alineen con los objetivos del negocio**. La economía es la **ciencia de elegir**, no la ciencia del dinero; la economía de la ingeniería es la rama aplicada de la microeconomía. Pregunta central: ¿conviene invertir los recursos limitados en esto, o rendirían más en otra cosa? Ingeniería es equilibrar lo **técnicamente posible** con lo **económicamente aceptable**. El software debe generar **valor**.

TEMAS Propuestas, flujo de caja, valor del dinero en el tiempo, comparación de alternativas, activos intangibles, y el proceso de decisión: entender el problema real → identificar soluciones factibles → definir criterios → evaluar → elegir → monitorear.

EJEMPLO ¿Programar el sistema de facturación desde cero ($30 000, 6 meses) o pagar uno existente ($200/mes)? Si eliges el primero «porque es más bonito», el SWEBOK te mira con decepción. Es lo que el profe pide cuando dice «¿.NET contra qué lo evaluaste?».

DÓNDE p. 288 (15-1) Introduction y figura de temas en p. 289

Práctica profesional · professional practice

SWEBOK Forma de prestar servicios cumpliendo ciertos estándares o criterios, tanto en el proceso como en el producto. Aplica a profesiones con cuerpo de conocimiento, **código de ética**, certificación y sociedades profesionales (ACM, IEEE).

DÓNDE p. 274 (14-1)

Seguridad de software · software security

SWEBOK Es mucho mejor **diseñar** la seguridad dentro del software que parcharla después. Hay que considerarla en cada etapa (requisitos, diseño, construcción, pruebas) y en el mantenimiento. Las vulnerabilidades también llegan por componentes de terceros (librerías, COTS, SO).

DÓNDE p. 264 (13-1)

SECCIÓN 7 · SLIDES 03A Y 01B · LO QUE SE CRUZA CON EL SWEBOK

Lo de clase que te pueden mezclar

El control es del SWEBOK, pero el profe repite que la taxonomía de Sommerville y las métricas son «preguntas garantizadas». Aquí lo mínimo para no confundir fuentes.

La trampa más probable: el SWEBOK divide los RNF en **restricciones de tecnología** y **calidad de servicio**. Sommerville (clase) los divide en **Producto / Organizacional / Externo**. Las dos son válidas; responde con la de la fuente que te pidan.

Sommerville	Subtipos	RNF modelo con métrica
Producto	**Eficiencia** (rendimiento, espacio) · **Usabilidad** · **Dependability** (disponibilidad, confiabilidad, robustez, reparabilidad, mantenibilidad) · **Seguridad**	«El 95 % de las consultas de notas responderá en ≤ 2 s con 3 000 usuarios concurrentes» (Producto · eficiencia · rendimiento).
Organizacional	**Ambiental** (plataforma, portabilidad) · **Operacional** (entrega, uso, interoperabilidad) · **De desarrollo** (lenguaje, método, estándares)	«El sistema funcionará en Windows 11 y Ubuntu 24.04 con ≤ 5 % de código dependiente de la plataforma» (Organizacional · ambiental).
Externo	**Regulatorio** · **Ético** · **Legislativo** (contable, safety, seguridad)	«El sistema cumplirá la Ley Orgánica de Protección de Datos Personales; el 100 % de los datos personales se cifrará en reposo» (Externo · legislativo).

Métricas que debes tener a mano (slides 23–30)

- **Rendimiento:** tiempo de respuesta promedio/máximo, transacciones por segundo, tiempo de refresco de pantalla
- **Espacio:** KB, número de chips de RAM
- **Usabilidad:** horas de capacitación, tiempo por tarea, errores por hora
- **Confiabilidad:** MTTF, probabilidad de no disponibilidad, tasa de fallas, % disponibilidad entre ciertas horas
- **Robustez:** tiempo de reinicio (MTTR), % de eventos que causan falla, probabilidad de corrupción de datos
- **Mantenibilidad:** tiempo para hacer un cambio simple/moderado/complejo
- **Portabilidad:** número de plataformas, % de sentencias dependientes de plataforma

Ejemplo de clasificación tipo examen

«El sistema estará disponible el 99 % del tiempo entre las 08h00 y la medianoche».
→ **Producto · dependability · availability**. Le falta: ¿para qué usuarios?, ¿cómo y en qué período se mide?

Actividades del proceso

Sommerville (4): especificación, desarrollo, validación, evolución.
Pressman (5): comunicación, planificación, modelado, construcción, despliegue.
SWEBOK: habla de procesos técnicos, de gestión técnica, organizacionales y de acuerdo (ISO 12207) p. 224.

Recuerda el estilo del profe: 2 o 3 líneas, sin relleno, sin inventar. Si citas otra fuente, nómbrala (p. ej., «según Sommerville…»). Si no sabes, mejor en blanco que «chueco».

SECCIÓN 8 · 5 MINUTOS QUE TE SALVAN PUNTOS

Parejas que se confunden

Pareja	La diferencia en una línea
Requisito de producto vs. de proyecto	Producto: qué hace/cumple el software. Proyecto: costo, plazo, personal del proyecto que lo construye.
Restricción de tecnología vs. calidad de servicio	Tecnología: nombra una tecnología (Java, Android). Calidad de servicio: un nivel de desempeño (2 s, 99 %).
Incompletitud vs. ambigüedad	Incompleto: falta información. Ambiguo: la información está, pero se entiende de varias formas.
Desarrollo vs. gestión de requisitos	Desarrollo: llegar a un acuerdo. Gestión: mantener el acuerdo en el tiempo.
Elicitación vs. análisis	Elicitar: sacar requisitos candidatos. Analizar: entender su significado, propiedades y conflictos.
Validación de requisitos vs. verificación	Validar: ¿son las necesidades verdaderas? (producto correcto). Verificar: ¿cumple lo especificado? (producto bien hecho).
Error vs. defecto vs. falla	Persona se equivoca → queda una imperfección en el software → el usuario ve que algo no funciona.
Scrubbing vs. scope matching	Scrubbing: quitar requisitos innecesarios y simplificar. Scope matching: que el alcance quepa en costo, plazo y personal.
Iterativo vs. incremental	Iterativo: ciclos que repiten y mejoran. Incremental: cada iteración **agrega** funcionalidad hasta completar.
Cascada vs. espiral	Cascada: predictiva y dirigida por documentos. Espiral: evolutiva y dirigida por riesgos.
Ágil vs. Scrum	Ágil es una mentalidad (valores y principios), no un método. Scrum es un método ágil.
Mantenimiento correctivo vs. preventivo	Correctivo: el problema ya apareció. Preventivo: lo arreglas antes de que aparezca.
Adaptativo vs. perfectivo vs. aditivo	Adaptativo: cambió el ambiente. Perfectivo: mejoras, documentación, refactor. Aditivo: funciones nuevas grandes.
Incertidumbre vs. riesgo	Incertidumbre: falta de información. Riesgo: su efecto en los objetivos (puede ser bueno o malo).
SWEBOK vs. Sommerville (RNF)	SWEBOK: tecnología / calidad de servicio. Sommerville: producto / organizacional / externo.
Def. de IS: SWEBOK vs. Sommerville	SWEBOK: sistemático, disciplinado, **cuantificable**; desarrollo, **operación** y mantenimiento. Sommerville: disciplina que cubre todos los aspectos de la producción de software, desde la especificación hasta el mantenimiento.

SECCIÓN 9 · TÁPATE LAS RESPUESTAS CON LA MANO

Simulacro de control de lectura

Formato real: definición corta + ejemplo. Intenta responder en voz alta antes de leer la respuesta modelo.

1. Según el SWEBOK, ¿qué es la ingeniería de software? Dé un ejemplo.

> Es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, la operación y el mantenimiento del software, es decir, aplicar la ingeniería al software (SWEBOK, p. 38). Ej.: desarrollar el sistema de citas de un hospital con requisitos validados, pruebas, control de versiones y métricas de disponibilidad, en vez de programarlo sin método.

2. ¿Qué es un requisito funcional y qué es uno no funcional? Un ejemplo de cada uno.

> RF: especifica comportamientos observables: políticas que el software hace cumplir y procesos que realiza (p. 46). Ej.: «el saldo de una cuenta nunca puede ser negativo». RNF: restringe las tecnologías de la implementación; se divide en restricciones de tecnología y de calidad de servicio (p. 46). Ej.: «la consulta de saldo responderá en ≤ 2 s».

3. ¿Cuáles son los dos problemas principales de requisitos según el SWEBOK?

> Incompletitud (hay necesidades o detalles que no se revelan a los ingenieros) y ambigüedad (el requisito admite varias interpretaciones y solo una es correcta) (p. 43). Ej.: «enviar la alerta pronto» es ambiguo; no mencionar que hay usuarios sin internet es incompleto.

4. ¿Qué es un stakeholder? Nombre 4 tipos.

> Persona, grupo u organización que participa en el proyecto, es afectada por su resultado o puede influir en él (p. 48). Tipos: clientes (pagan), usuarios, entes reguladores, expertos del dominio. (También: personas perjudicadas si el proyecto tiene éxito).

5. ¿Qué es la elicitación y nombre 4 técnicas?

> Actividad cuyo objetivo es hacer aflorar requisitos candidatos; reduce la incompletitud y no es pasiva (p. 48). Técnicas: entrevistas, talleres JAD, cuestionarios, prototipos, observación, apprenticing, user story mapping (p. 49).

6. Explique el filtro de la tecnología perfecta con un ejemplo.

> Si existiera una computadora infinitamente rápida, con memoria ilimitada, gratis y sin fallas, lo que todavía habría que pedir es funcional; todo lo demás es no funcional (p. 47). Ej.: «registrar una reserva» sigue siendo necesario (RF); «responder en 1 s» deja de importar (RNF).

7. ¿Qué es la validación de requisitos y cuáles son sus 3 métodos?

> Ganar confianza en que los requisitos representan las verdaderas necesidades de los stakeholders (p. 57). Métodos: revisiones, simulación y ejecución, y prototipado (p. 57–58). Ej.: mostrar un prototipo de la app al dueño del restaurante para confirmar el flujo del pedido.

8. ¿Qué es software quality en el contexto de la ingeniería de software?

> El grado en que un producto de software cumple los requisitos establecidos, considerando que la calidad depende de qué tan bien esos requisitos representan las necesidades y expectativas de los stakeholders (p. 248). Ej.: un sistema que cumple todo lo pedido, pero lo pedido no era lo que el usuario necesitaba, no tiene calidad.

9. ¿Qué es software configuration management?

> Aplicar la gestión de configuración durante todo el ciclo de vida para asegurar que los ítems de configuración estén completos y correctos: identificarlos, controlar sus cambios, registrar su estado y verificar su cumplimiento (p. 185). Ej.: usar Git con versiones etiquetadas y una línea base del ERS que solo cambia con una solicitud formal.

10. ¿Qué es software engineering economics?

> Área que alinea las decisiones técnicas con los objetivos del negocio; la economía es la ciencia de elegir y la pregunta clave es si invertir aquí los recursos limitados da más valor que otra alternativa (p. 288). Ej.: comparar desarrollar un sistema propio contra comprar uno existente según costo y beneficio.

11. Diferencie error, defecto y falla con un ejemplo.

> Error: acción humana con resultado incorrecto. Defecto: imperfección en el producto que no cumple su especificación. Falla: el sistema no puede realizar una función requerida; ocurre al ejecutarse el defecto (p. 248). Ej.: escribir > en vez de >= (error), la línea queda en el código (defecto), un estudiante con 6 aparece reprobado (falla).

12. ¿Qué es el mantenimiento de software? Nombre 3 tipos con ejemplo.

> La totalidad de actividades para dar soporte costo-efectivo al software en operación (p. 167). Correctivo: arreglar un bug reportado. Adaptativo: adaptar a una nueva versión del sistema operativo. Perfectivo: refactorizar para mejorar el rendimiento (p. 170–171).

13. ¿Qué dice el SWEBOK de Scrum?

> Es el enfoque ágil más orientado a la gestión de proyectos: el Scrum Master gestiona las actividades, cada incremento es un sprint de máximo 30 días, el Product Owner define el product backlog, en cada sprint se prueba y libera una versión funcionando y el Daily Scrum mantiene el trabajo según el plan (p. 242).

14. Nombre 8 áreas de conocimiento del SWEBOK.

> Requisitos, Arquitectura, Diseño, Construcción, Pruebas, Operaciones, Mantenimiento, Gestión de configuración (y además: Gestión, Proceso, Modelos y métodos, Calidad, Seguridad, Práctica profesional, Economía, y 3 de fundamentos).

15. ¿Qué propiedades debe tener un requisito individual según el SWEBOK? Nombre 5.

> No ambiguo, verificable (cuantificado), vinculante, atómico, que represente necesidades reales, con vocabulario del stakeholder y aceptable para todos (p. 50). Ej. de requisito que falla: «el sistema será rápido» (ambiguo y no verificable).

SECCIÓN 10 · ÚLTIMO REPASO ANTES DE ENTRAR

Chuleta de 1 página

Ingeniería de software · p. 38
Enfoque **sistemático, disciplinado y cuantificable** para **desarrollar, operar y mantener** software.

Software · p. 38
Programas + procedimientos + documentación + datos.

18 KAs · p. 38
Nuevas en v4: Arquitectura, Seguridad, Operaciones. Ágil y DevOps en casi todas.

Requisito · p. 44–45
Propiedad que el software debe exhibir para resolver un problema real.

2 problemas · p. 43
Incompletitud (falta info) y **ambigüedad** (varias interpretaciones).

Categorías · p. 45–46
Proyecto vs. producto → funcional vs. no funcional → tecnología vs. calidad de servicio.

RF · p. 46
Comportamientos observables: **políticas** y **procesos**.

RNF · p. 46
Restringen tecnologías: **tecnología con nombre** o **nivel de desempeño**.

Filtro perfecto · p. 47
Computadora infinita: lo que aún pides es RF.

Derivado · p. 48
Lo impone el equipo interno, no un stakeholder externo.

Actividades · p. 48
Desarrollo (elicitar, analizar, especificar, validar) = acuerdo. Gestión (scrubbing, cambios, alcance) = mantenerlo.

Stakeholder · p. 48
Participa, es afectado o influye.

Elicitación · p. 48–50
Hacer aflorar requisitos candidatos. Entrevistas, JAD, cuestionarios, prototipos, observación, apprenticing.

Propiedades · p. 50
No ambiguo, verificable, vinculante, atómico, real. Conjunto: completo, conciso, consistente, factible.

5 porqués · p. 50
Preguntar «¿por qué?» hasta llegar al problema real.

Falla / perfección · p. 50–51
Rango de valor de un RNF; elegir el nivel más costo-efectivo.

Conflictos · p. 52
Negociar o familia de productos (invariantes vs. variantes).

Especificación · p. 52–56
Natural libre, estructurada (actor-acción, casos de uso, historias), por aceptación (ATDD/BDD), por modelos.

Validación · p. 57–58
¿Son las necesidades reales? Revisiones, simulación, prototipos.

Priorizar · p. 60
Kano: pesa también cuánto enoja que falte.

Trazabilidad · p. 60–61
Consistencia + análisis de impacto, hacia atrás y adelante.

Calidad · p. 248
Grado en que cumple requisitos que representan bien las necesidades.

Error → defecto → falla · p. 248
Persona → software → usuario lo ve.

V&V · p. 257
Verificar: bien hecho. Validar: lo correcto.

Proceso · p. 223
Actividades que transforman entradas en salidas.

Ciclo de vida · p. 223
De la concepción al retiro.

Modelos · p. 226
Cascada (predictivo, por documentos), espiral (por riesgos), prototipado, UP/RUP (iterativo e incremental).

Ágil · p. 226–227, 241
Abierto al cambio; no es un método; sí documenta. Ciclos cortos, PDCA.

Scrum · p. 242
Sprint ≤ 30 días, SM, PO, backlog, daily.

SCM · p. 185
Identificar, controlar cambios, registrar estado, verificar los CIs. Línea base p. 191.

Mantenimiento · p. 167–171
Soporte costo-efectivo. Correctivo, preventivo, adaptativo, aditivo, perfectivo, emergencia.

Gestión y riesgo · p. 202, 210
Planificar, estimar, medir, controlar. Riesgo = efecto de la incertidumbre (+ o −).

Economía · p. 288
Ciencia de elegir; alinear técnica con negocio.

Definición + para qué sirve + ejemplo tuyo. 3 líneas. Si no sabes, en blanco. Respira. Ya leíste más del SWEBOK que el 90 % del curso. 💀🍷